

Seurat v5 + BPCells — Full Pipeline Workflow Reference

Purpose

This document describes the two recommended end-to-end snRNA-seq pipelines using Seurat v5 and BPCells: **Workflow A (SCTransform)** and **Workflow B (LogNormalize)**. For each step it explains which function to use, why it is correct for a v5/BPCells pipeline, and why the Assay v3 counterpart fails or is insufficient.

For integration-specific rationale (v4 vs v5 API differences, RAM table, **IntegrateLayers** parameter guide) see `seurat_v5_sct_integration.md`.

The Central Architectural Distinction

	Assay v3 (Assay / SCTAssay)	Assay v5 (Assay5)
Storage	Fixed S4 slots: <code>@counts</code> , <code>@data</code> , <code>@scale.data</code>	Named layers: any number, any name
Slot type validation	S4 <code>checkSlotAssignment</code> enforces dgCMatrx	<code>LayerData<-</code> accepts IterableMatrix (BPCells)
BPCells compatible	No — <code>slot<-</code> rejects MatrixDir at the type-check level	Yes — designed for on-disk IterableMatrix
Multi-sample	One concatenated matrix	One layer per sample: <code>counts.sample1</code> , <code>counts.sample2</code>
Memory model	Always fully in RAM	On-disk; R holds a lazy pointer, not the data

Why BPCells only works with Assay5: BPCells matrices are instances of **IterableMatrix**. Assay v3's S4 slot validator requires **dgCMatrx** — it rejects **IterableMatrix** with a type error. Assay5's `LayerData<-` setter explicitly accepts **IterableMatrix**, so the matrix stays on disk.

Any Seurat operation that internally calls `slot<-` on an Assay v3 object (such as `RenameCells()` called by `merge()`) will fail with a type error if BPCells data is present, even if the initial write succeeded via an `attr<-` bypass.

Choosing a Workflow

Comparison: SCTransform vs LogNormalize

Criterion	Workflow A — SCTransform	Workflow B — LogNormalize
Normalization model	Negative-binomial GLM per gene; removes sequencing-depth confounding via Pearson residuals	$\text{Log}(\text{count} / \text{total} \times 10\,000)$; assumes roughly Poisson count distribution
Mean-variance handling	Explicitly models overdispersion — better for snRNA-seq (low UMI/nucleus)	Assumes Poisson variance; may over-retain highly-expressed genes as HVGs
BPCells compatibility	Indirect: SCT fails on BPCells; requires per-sample execution + conversion hack	Direct: all steps stream through BPCells on-disk; no conversion needed
Assay produced	SCTAssay (v3) → converted to Assay5 ; loses <code>@SCTModel.list</code>	Assay5 throughout; no conversion, no data loss

Criterion	Workflow A — SCTransform	Workflow B — LogNormalize
PrepSCTFindMarkers()	Unavailable after <code>@SCTModel.list</code> is discarded	Not applicable (LogNormalize DE is <code>FindMarkers()</code> directly)
DE after integration	<code>FindMarkers()</code> on SCT counts (approximate)	<code>FindMarkers()</code> on RNA normalised counts (standard)
RAM strategy	Per-sample SCT → BPCells → merge; peak RAM bounded to one sample	Direct merge → normalize on merged; RAM scales with total cells
Pipeline complexity	Higher: per-sample loop + conversion function + merge	Lower: merge → normalize → integrate in sequence
Computational cost	Higher (GLM fitting per gene per sample)	Lower (log ratio per cell)
Official BPCells support	Not officially supported; workaround required	Officially supported and documented by Seurat team

When to choose SCTransform (Workflow A)

- snRNA-seq data (brain, nucleus-only): lower UMI counts per nucleus make depth correction more important. SCTransform’s NB regression is theoretically better suited to overdispersed, sparse data.
- You are comparing rare cell populations where false HVGs from depth confounding could mask biological signal.
- You accept the `@SCTModel.list` loss (no `PrepSCTFindMarkers`) and the per-sample pipeline complexity.
- An official SCT5 class becomes available in a future Seurat release — at that point this workflow becomes the clear choice.

When to choose LogNormalize (Workflow B)

- You need a fully supported, stable pipeline with no API workarounds.
- RAM during the merge step is a concern and you prefer simplicity over theoretical optimality.
- Your dataset has reasonably uniform sequencing depth across cells, reducing the advantage of NB regression.
- You are doing bulk comparisons (WT vs KO) rather than rare cell-type discovery where HVG selection precision matters most.
- You want reliable `FindMarkers()` DE results without worrying about SCT model availability.

Workflow A — SCTransform + BPCells (Assay5)

Step A1 — Load Raw Data as BPCells

Functions: `BPCells::open_matrix_10x_hdf5()` → `BPCells::write_matrix_dir()` → `BPCells::open_matrix_dir()` → `CreateSeuratObject()`

Loads the count matrix from a CellBender/CellRanger HDF5 file as a lazy `IterableMatrix`. `write_matrix_dir()` serialises it to a compressed bit-packed directory on disk. `CreateSeuratObject()` with an `IterableMatrix` argument automatically creates an **Assay5** RNA assay. Cell metadata (`nCount_RNA`, `nFeature_RNA`) is computed lazily without loading the matrix.

Why not `Read10X()` + `CreateSeuratObject()`: `Read10X()` loads the full count matrix as a `dgCMatrx` into RAM immediately — the bottleneck on a 16 GB machine with large datasets.

Step A2 — Quality Control

Functions: `PercentageFeatureSet()`, `VlnPlot()`, `subset()`

`PercentageFeatureSet()` works on `Assay5` with `BPCells` by streaming chunks — only a small portion of the count matrix is in RAM at any time. `subset()` reindexes the `BPCells` layer to retained barcodes without rewriting the on-disk files.

For doublet detection (`scDblFinder()`): `BPCells` must be materialised to `dgCMatrx` per sample before passing to `SingleCellExperiment`, since `SCE` does not accept `IterableMatrix`. Done one sample at a time — safe at 16 GB.

Why not `Assay v3 subset()`: Internally calls `slot<-` on the assay, which triggers `checkSlotAssignment` and rejects `IterableMatrix`.

Step A3 — SCTransform Normalisation (per sample)

Function: `SCTransform()` with `vst.flavor = "v2"`, `method = "glmGamPoi"`

Why per sample: `SCTransform()` fails on `BPCells`-backed `Assay5` objects. Internally it calls `dim(x) <- length(x)` to reshape matrix data — invalid for a lazy, read-only `IterableMatrix`. Per-sample execution bounds peak RAM to one library.

What `SCTransform()` produces: An `SCTAssay` object (`Assay v3` subclass). `SCTAssay` carries `@SCTModel.list` — per-cell fitted NB regression parameters (depth offset, dispersion, gene-specific coefficients). Three data layers:

- `scale.data` — Pearson residuals (non-sparse; variable genes only). PCA input.
- `counts` — depth-corrected UMI counts.
- `data` — log-normalised corrected counts (for visualisation).

Why `SCTAssay` is not `Assay5`: `@SCTModel.list` has no equivalent slot in `Assay5`. Converting discards it, which breaks `PrepSCTFindMarkers()` and reduces accuracy of anchor rescoring in `IntegrateLayers(normalization.method = "SCT")`. This is an active Seurat GitHub limitation (issues #7814, #9707).

Step A4 — Offload SCT to BPCells (per sample)

Function: `sct_to_assay5_bpcells()` (project-internal, `code/02_sc_functions.R`)

Coerces `SCTAssay` → `Assay5`, writes `counts` and `data` to `BPCells` on-disk directories via `write_matrix_dir()`, then reassigns those layers via `LayerData<-` (valid because `Assay5` accepts `IterableMatrix`). Idempotent: if the `BPCells` directories exist the data is reopened, not rewritten.

Trade-off: `@SCTModel.list` is discarded. Use `FindMarkers()` directly on the SCT `counts` layer for DE testing; `PrepSCTFindMarkers()` is unavailable.

Why not `sct_to_bpcells()`: Uses `attr<-` to bypass S4 validation at write time. Breaks on any subsequent `slot<-` call (e.g. `merge()` → `RenameCells()`).

Step A5 — Merge

Function: `merge()` — single call with `y = list(obj2, ..., objN)`

A single `merge()` call is mandatory. Iterative merging accumulates `.1/.2/...` suffixes at every iteration. A single call produces clean layer names: `counts.sample1`, `data.sample1`, etc.

`merge()` calls `RenameCells()` → `slot<-` on each assay. This is why the SCT assay must be **Assay5** by this point: Assay5's `slot<-` targets the layer index (`list`), not a typed S4 slot, so `IterableMatrix` is accepted.

Step A6 — PCA

Function: `RunPCA()` with `assay = "SCT"`

PCA runs on `scale.data` (Pearson residuals) — already variance-stabilised and mean-centred by `SCTransform`. No additional `ScaleData()` is needed.

Why not `ScaleData()` before `RunPCA()`: `ScaleData()` would overwrite the Pearson residuals with z-scores of log-normalised data — wrong input for the SCT path.

Step A7 — Integration

RPCA — `IntegrateLayers()` with `method = RPCAIntegration` `normalization.method = "SCT"` instructs `IntegrateLayers` to use the SCT model during anchor rescoring (equivalent to the v4 `PrepSCTIntegration` step, now internalised). `orig.reduction = "pca"` provides the starting embedding.

Output: reduction `integrated.rpca`. No new assay is created.

Why not v4 `FindIntegrationAnchors()` + `IntegrateData()`: `IntegrateData` stores a full corrected expression matrix and builds the anchor weight matrix for all pairs simultaneously — exhausts RAM on large datasets.

Harmony — `IntegrateLayers()` with `method = HarmonyIntegration` Iteratively adjusts PCA embeddings to remove batch variance while preserving biological variance. Faster than RPCA for very large datasets.

Output: reduction `harmony`. Use `reduction = "harmony"` in downstream steps.

Step A8 — Join Layers

Function: `JoinLayers()` on the SCT assay

Collapses per-sample layers into single `counts` and `data` layers. Required before `FindMarkers()`, `DotPlot()`, `FeaturePlot()`. Assay5-only — no v3 equivalent exists because v3 always stored a single concatenated matrix.

Note: materialises BPCells layers into RAM on join — plan for the RAM cost.

Steps A9–A11 — Clustering and Visualisation

Functions: `FindNeighbors()`, `FindClusters()`, `RunUMAP()`, `DimPlot()`

All operate on the integrated reduction (`integrated.rpca` or `harmony`), not on the assay. Assay version is irrelevant. `RunUMAP()` must use the integrated reduction — using `"pca"` would visualise unintegrated variance.

Workflow B — LogNormalize + BPCells (Assay5)

This is the officially supported BPCells pathway. The RNA assay stays as Assay5 with BPCells on disk throughout the entire pipeline — no conversion, no workarounds.

Step B1 — Load Raw Data as BPCells

Identical to Step A1. Output: RNA assay (Assay5 + BPCells on disk).

Step B2 — Quality Control

Identical to Step A2. BPCells streaming makes `PercentageFeatureSet()` and `subset()` RAM-efficient throughout.

Step B3 — Merge

Function: `merge()` — single call with `y = list(obj2, ..., objN)`

Merge all per-sample singlet objects in one call. The resulting RNA Assay5 has one layer per sample: `counts.sample1`, `counts.sample2`, etc.

This is simpler than Workflow A because there is no per-sample normalisation loop — the raw BPCells-backed objects are merged directly and normalisation happens on the merged, split-layer object in the next step.

Step B4 — Normalisation

Function: `NormalizeData()` with `normalization.method = "LogNormalize"`

Seurat v5 detects the split layers and applies log-normalisation independently per sample layer, writing the result back to a `data.<sample>` layer in each slot. This is entirely streaming over BPCells — no data is loaded into RAM. The result is equivalent to normalising each sample separately before merging.

Why this works with BPCells: `NormalizeData()` reads counts in chunks via the BPCells streaming API, computes per-cell size factors, and writes normalised values back to disk. The Assay5 layer structure keeps everything on-disk.

Why not `SCTransform()` here: It fails on BPCells matrices (see Step A3). The single-merged-object `SCTransform()` call (the Seurat v5 official workflow) also fails for the same reason.

Step B5 — Identify Variable Features

Function: `FindVariableFeatures()` with `nfeatures = N_INTEGRATION_FEATURES`

In Seurat v5, `FindVariableFeatures()` on a split-layer object identifies HVGs per sample and then computes a consensus set across samples (mean of per-sample ranks). This prevents HVG selection being dominated by any single sample. Operates via BPCells streaming — no materialisation.

Why not running per-sample separately: The split-layer approach gives the same result as running per-sample and intersecting, but in one call. It is the v5-recommended path.

Step B6 — Scale Data

Function: `ScaleData()` with `features = VariableFeatures(obj)`

`ScaleData()` z-scores each variable gene (mean = 0, variance = 1) and stores the result in the `scale.data` layer. Only the variable genes (~2000) are scaled — this is the only step that loads data into RAM, but the subset of variable genes is small enough to be manageable.

`scale.data` is stored in-memory (it is a dense non-sparse matrix). This is unavoidable: PCA requires a dense input and `BPCells` cannot store dense matrices on disk.

Why `ScaleData()` is needed here but not in Workflow A: `SCTransform` produces Pearson residuals that are already mean-centred and variance-stabilised — they are the `scale.data` analogue. `NormalizeData()` only log-scales counts and does not centre or variance-stabilise, so `ScaleData()` is required before PCA.

Step B7 — PCA

Function: `RunPCA()` with `assay = "RNA"`

PCA runs on the `scale.data` layer of the RNA assay (the scaled HVGs). Same function as in Workflow A; the difference is the input assay and the fact that `scale.data` was produced by `ScaleData()` rather than `SCTransform`.

Step B8 — Integration

RPCA — `IntegrateLayers()` with `method = RPCAIntegration` `normalization.method = "LogNormalize"` (the default — no special argument needed). `orig.reduction = "pca"`. Output: `reduction integrated.rpca`.

Anchor finding uses the split layers (one per sample) to build pairwise projections — this is what `IntegrateLayers()` was architecturally designed for. No RAM spike from building a full weight matrix.

Harmony — `IntegrateLayers()` with `method = HarmonyIntegration` `group.by.vars = "sample_id"`, `normalization.method = "LogNormalize"`. Output: `reduction harmony`.

Both RPCA and Harmony work natively and identically to Workflow A at the integration step — the difference is upstream (what normalisation was done).

Step B9 — Join Layers

Function: `JoinLayers()` on the RNA assay

Same purpose as Step A8 — collapses per-sample layers into single `counts` and `data` layers. Required before DE and most visualisation functions.

Steps B10–B12 — Clustering and Visualisation

Identical to Steps A9–A11 in function and rationale. Use integrated reduction in `FindNeighbors()`, `FindClusters()`, and `RunUMAP()`.

Summary Tables

Function–Assay Compatibility

Function	Assay v3 + BPCells	Assay5 + BPCells
<code>CreateSeuratObject()</code>	Creates Assay5 regardless	Creates Assay5
<code>PercentageFeatureSet()</code>	Fails (<code>slot<- path</code>)	Works (streaming)
<code>subset()</code>	Fails (<code>slot<- path</code>)	Works
<code>NormalizeData()</code>	N/A	Works (streaming)
<code>FindVariableFeatures()</code>	N/A	Works (streaming)
<code>ScaleData()</code>	N/A	Works (materialises HVGs only)
<code>SCTransform()</code>	Fails on BPCells input	Fails on BPCells input
<code>merge()</code>	Fails if BPCells present	Works
<code>RunPCA()</code>	Works (reads <code>scale.data</code>)	Works
<code>IntegrateLayers()</code>	N/A	Works
<code>JoinLayers()</code>	Does not exist	Works
<code>FindNeighbors()</code>	Works (reduction only)	Works (reduction only)
<code>FindClusters()</code>	Works	Works
<code>RunUMAP()</code>	Works	Works
<code>FindMarkers()</code>	Works	Works
<code>PrepSCTFindMarkers()</code>	Works (if <code>@SCTModel.list</code> present)	Not applicable

Pipeline Comparison

Step	Workflow A (SCT)	Workflow B (LogNorm)	Assay at this step
Load	<code>open_matrix_* + CreateSeuratObject</code>	Same	RNA (Assay5 + BPCells)
QC	<code>PercentageFeatureSet, subset</code>	Same	RNA (Assay5 + BPCells)
Normalise	<code>SCTransform</code> per sample	<code>NormalizeData</code> on merged	SCT Assay v3 / RNA Assay5
Offload	<code>sct_to_assay5_bpccells</code>	Not needed	SCT (Assay5 + BPCells) / RNA unchanged
Variable features	(done by <code>SCTransform</code>)	<code>FindVariableFeatures</code>	—
Scale	(done by <code>SCTransform</code>)	<code>ScaleData</code>	<code>scale.data</code> in RAM (HVGs only)
Merge	<code>merge</code> (single call)	<code>merge</code> (single call)	SCT Assay5 / RNA Assay5
PCA	<code>RunPCA</code> on SCT	<code>RunPCA</code> on RNA	reduction: <code>pca</code>
Integration	<code>IntegrateLayers(normalize.method="LogNormalize")</code>	<code>IntegrateLayers(normalize.method="LogNormalize")</code>	<code>integrated.rpca / harmony</code>
Join layers	<code>JoinLayers</code>	<code>JoinLayers</code>	unified layers in RAM
Clustering	<code>FindNeighbors + FindClusters</code>	Same	graph + cluster labels
UMAP	<code>RunUMAP</code>	Same	reduction: <code>umap</code>
DE	<code>FindMarkers</code> on SCT counts	<code>FindMarkers</code> on RNA data	—

Known Limitations

Workflow A

@SCTModel.list loss. Converting `SCTAssay` $\rightarrow$ `Assay5` discards the per-cell regression model. `PrepSCTFindMarkers()` cannot be used; use `FindMarkers()` on the SCT counts layer directly. Anchor rescoring in `IntegrateLayers(normalization.method = "SCT")` uses an approximation. Tracked in Seurat GitHub issues #7814 and #9707.

SCTransform fails on BPCells. Running `SCTransform()` on any BPCells-backed object fails with `dim(x) <- length(x)` — the function tries to reshape the lazy matrix, which is invalid. Per-sample execution is the only viable path.

Workflow B

Poisson assumption. Log-normalisation does not explicitly model overdispersion. On snRNA-seq data with low UMI counts per nucleus, highly expressed genes may inflate the HVG list due to higher technical variance that the model treats as biological.

ScaleData() RAM spike. Even though only variable genes are loaded, on very large datasets (> 500k cells) `scale.data` for 2000 genes can reach several GB. Reduce `N_INTEGRATION_FEATURES` if needed.

References

Resource	URL
Seurat v5 BPCells vignette	https://satijalab.org/seurat/articles/seurat5_bpccells_interaction_vignette
Seurat v5 integration vignette	https://satijalab.org/seurat/articles/seurat5_integration
SCTransform vignette	https://satijalab.org/seurat/articles/sctransform_vignette
BPCells package	https://bnprks.github.io/BPCells
SCT + BPCells compatibility issue	https://github.com/satijalab/seurat/issues/7814
SCT + multilayer issue	https://github.com/satijalab/seurat/issues/9707
Companion doc: integration rationale	<code>seurat_v5_sct_integration.md</code>